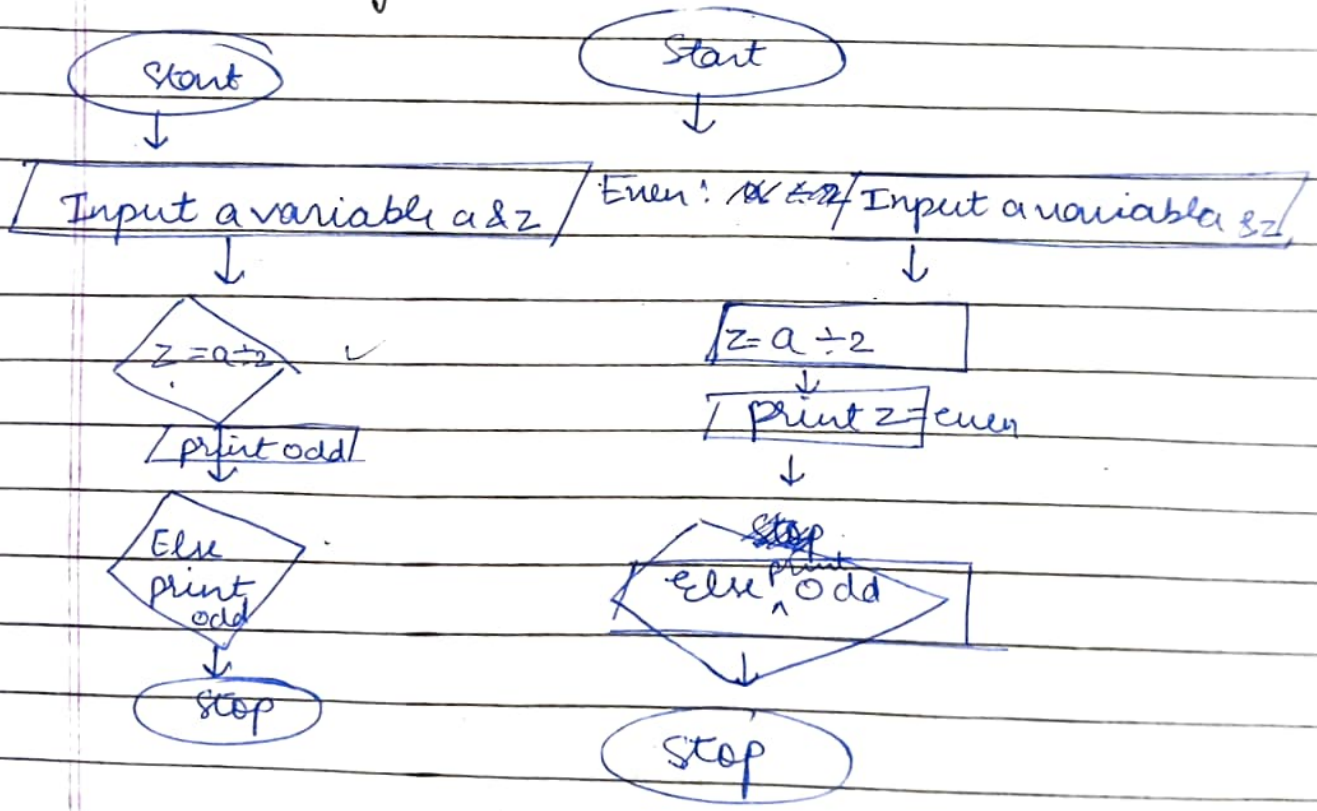
abc.c

C program → compiler → object code

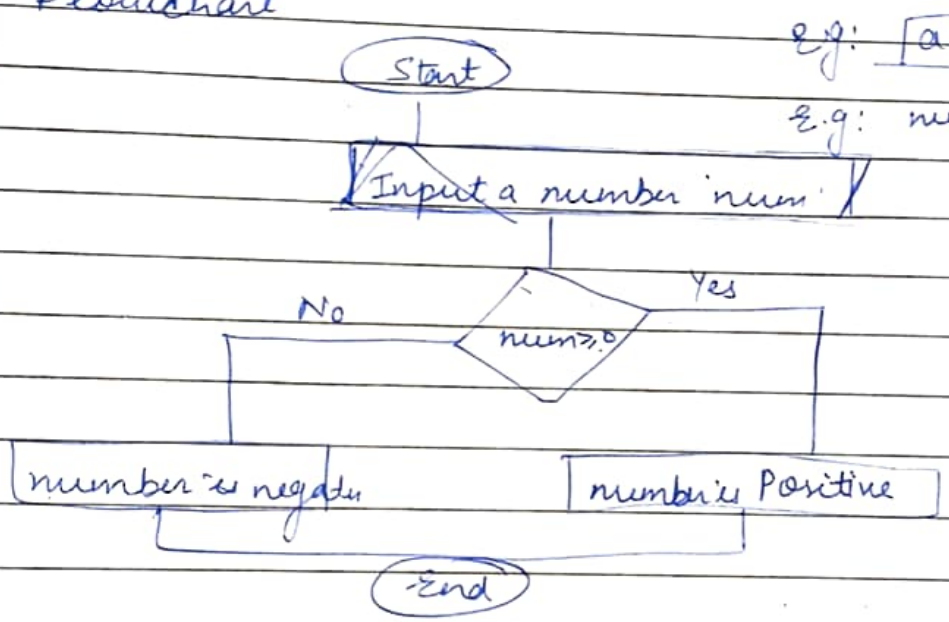
Write an algorithm of a number is even/odd:



(c) Write an algorithm & flowchart to determine the no. is positive or negative

S-1 Start
S-2 Input a number num
S-3. Assign value to variable N
S-4. Check if N is ~~greater~~ then or equal to ~~zero~~ 0.
S-5. If true print N is positive
S-6. If false print N is negative
S-7. Stop

Flowchart

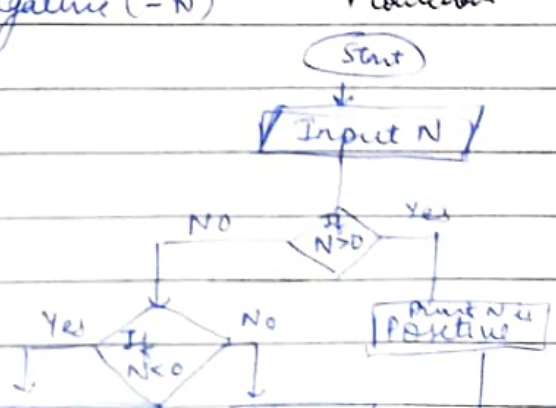


Q. Write an algorithm & flowchart given a number is +ve, -ve or 0.

Algo

S-1 Start
S-2 Read / Take Input a number N
S-3 If N > 0, then print "positive" (+N)
S-4 Else if N < 0 then print "negative (-N)
S-5 Else N = 0 print "zero"
S-6 Stop / End

Flowchart



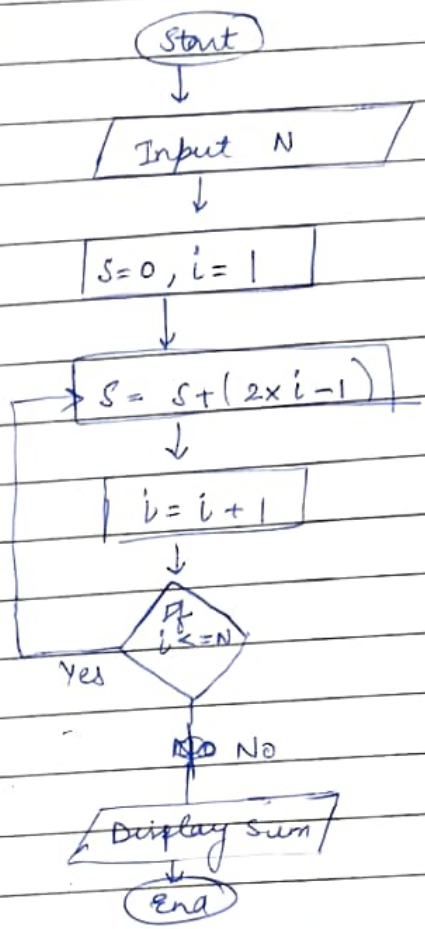
$1+3+5+\cdots+2n-1 = n^2$

Q. Write an algorithm & flowchart to develop the following formula $S = 1+3+5+\cdots+n$

Algorithm

S-1 Start
S-2 Take a variable 'n'
S-3 $S=0$, $i=1$
S-4 $S = S+(2\times i-1)$
S-5 $i = i+1$
S-6 if $i <= n+1$ then
S-7 Go to Step 4
S-8 Else
S-9 Display sum
End

Flowchart



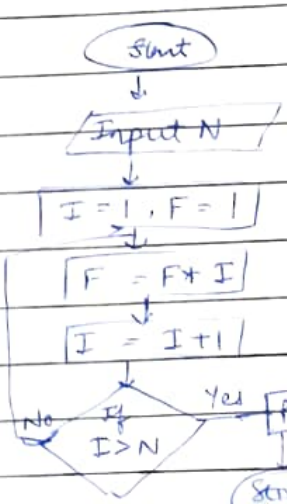
N=5

S	i
0	1
1	2
4	3
9	4
16	5
25	6

Q.1 Write an algorithm & flowchart to determine a factorial n
Q.2 Write an algorithm & flowchart to determine a prime no

1. Algo
S-1 Start
S-2 Input a number N
S-3 Initialize I=1, F=1
S-4 F = F*I
S-5 I = I+1
S-6 while I<=N Repeat step 4 to 6
S-7 print F
S-8 End

Flowchart



C language tokens

→ identifiers
→ keywords
→ string
→ constants
→ special characters
→ operators

⇒ Identifiers - ① It indicates names given to a progra element such as variable array function.
② The first character in an identifier must be an alpha -bet or an underscore & can be followed only by any no. alphabets or digits or underscore.
③ They must not begin with the digits.

① std in
② std out
③ std err

4. Uppercase & lowercase letters are distinct.
5. Identifiers are case sensitive.
6. Commas & blank spaces are not allowed with an identifier.
7. Keywords cannot be used in identifier
8. Identifier should not be of more than 31 characters
9. Identifier must be meaningful, short, quickly & easily typed & easily read
10. It is of two types internal (local), external (global)

int age, "Here age → identifier
float salary, → salary → identifier
int price, price → identifier
double interest, → interest → identifier

Valid Identifier	Invalid Identifier
age	1marks
student_name	student name
marks1	float → keyword
total	

1. Variables: (Being able to change)

2. Data type - It tells the compiler what kind of data a variable is going to store char grade='A'; int a = 10; float pi = 3.14; char a =

Data type
numeric
No with decimal point
with decimal point

3. primitive data type
- float
  - float
  - double
  - long double
- char
  - signed
  - unsigned
- int
- int
  - int
  - long int
  - unsigned int
  - long long int
  - unsigned long long int
  - short int
  - Unsigned short int

Fundamental → int, float, char

User defined Data Type → structure, union, enumeration, typedef

Derived data type → array, pointer, string

③ keywords:

④ Range:

Type	Range (from	to	Byte	Represents
char/ short	-128	128	1 byte	characters
unsigned char	0	255	1	characters
int	-32768	+32767	2	whole no
unsigned int	0	65535	2	whole no
long	-2,147,438,648	2,147,438,647	4	
unsigned long	0	4,294,967,295	4	
float	3.4×10^{-38}	3.4×10^{38}	4	fractional no
double	$+1.7 \times 10^{-308}$	$1.7 \times 10^{+308}$	8	fractional no
long double	3.4×10^{-4932}	3.4×10^{4932}	10	fractional no

① Expression - An expression is a combination of operands & operators that produces a value.

E.g: a + b → operand-2, operator; operand-1

, a * b, a / b, a % b, a - a

, a * b + 10, a - b * 10

② Statement - It is an instruction that performs an action in C program.

Eg: x = 10;

printf("%d", x);

sum = a + b;

③ Symbolic constant - A symbolic constant is a name given to a constant value instit of using the value directly throughout the program we use the meaningful name.

E.g: Area = πr^2

area = 3.14 * r * r ⇒ pi * r * r

⇓

define float pi = 3.14;

```
include <stdio.h>
int main ( ) {
float r, area ;
float pi = 3.14;
printf ("Enter the value of radius: ");
scanf ("%f", &r);
Area = pi * r * r ;
printf ("Area of circle is : ", area);
return 0;
}
```

④ Input / output
- printf
- scanf — Input one character
- getchar — Reads one character
- putchar() — Output one character

char ch
ch = getchar()
ch = 'A'
putchar(ch);

Unary
1 opera
Eg: +a,
-2

puts → used to O/p a string, char name [] = "Abhay"; O/p → Abhay
puts (name);

⑤

Function -	Description
printf	Used for output
Scanf	Takes input from user
getchar	Read one character
putchar	Output one character
puts	used to O/p a string

⑤ Fuction : A function is a block of code which only runs when it is called. we can pass data know as parameter into a function.

```
#include<stdio.h>
void main() {
int a = 10, b = 20, ans;
ans = MCA ( a, b);
printf ("%d", ans);
}
MCA (int x, int y);
int add;
add = x+y;
return 0;
}
```

⑥ Preprocessor commands : ~~preprocessor directives~~ These instructions processed before the actual ~~compilation~~ compilation of c program

Operators & Expression :

① Arithmetic operator (+, −, /, %, *)

Eg: a = 10;
b = 3,
a+b = 13
a − b = 7
a * b = 30
a / b = 3
a % b = 1

Unary
1 operator
Eg: ++2 , ++ -2

Binary
2 operator
Eg: 2+3

Ternary
3 operator
exp1 ? exp2 : exp3
(10 > 7) ? (10+20)

② Assignment operator (=)

```
#include <stdio.h>
void main ( ) {
int a = 12;
   a = 200;
printf
Return 0
```

NOTE: expression is not allowed on the left side (b+c=20)

	Left value	Right value
a=20	variable	literal/constant → a=20
b+c=20	expression	variable sum=
20=a	literal/constant	expression sum=a+b

③ Unary operator: operator working with one operand

++i,

--i,

-i,

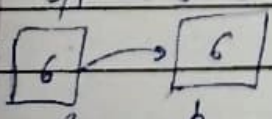
Modify operator

- increment operator
  - Pre increment (++) variable
  - post increment (--) variable
- Decrement operator
  - Pre decrement (--) variable
  - post decre variable

Pre increment

```
int a = 5 , b;
b = ++ a;
printf ("%d %d ", a, b);
```

O/P → 6 6

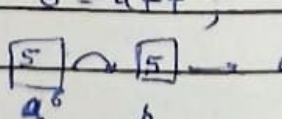


post increment

① first use

② then increment

b = a++;



Logical

Bdmin

Library

Objectives

Logical
Bitwise
Conditional operator
library f"

conditional statement

if - else
while
do while
for statement
nested - loops

Relational operators

>
<
>=
<=
==
!=

Binary operator

Result of any relational operator is always '0' or '1'

E.g: printf("%d", 10<20); o/p → 1
printf("%d", 10<1); o/p → 0

Top to bottom

*/ %
+ -
< > <= >=
== !=
=

Left to Right

? R to L ?

Logical operator

Logical AND (&&)

a	b	a&&b
T	T	T
T	F	F
F	T	F
F	F	F

NOTE:- If both operators are non zero then only result will be 1 otherwise it is 0

printf("%d", 2&&-5); ⟶ (1)

Logical OR (||)

Result is '1' if atleast 1 operand is non zero otherwise '0'

e.g: !(2.75)
⇩
!(1)
0 (False)

③ Logical NOT (!)

NOT(TRUE) = !(TRUE) = False
NOT(False) = !(False) = True

Q. int a;
a = 2<5 != 2<5 && 8>10 || = 8>10;
Printf("%d", a);

T
F
F

Q. int a;
a = 2 != 5 == 3 && 6>10 == 10>6;

F
F

1 & 0 ⇒ 0	
0 & 1 ⇒ 0	
0 & 0 ⇒ 0	
1 & 1 = 1	

Bitwise Operator

1 Bitwise AND (&)
2 Bitwise OR (|)
3 Bitwise XOR (^)
4 Bitwise NOT (~)
5 Bitwise left shift (<<)
6 Bitwise right shift (>>)

e.g ① int a = 5, b = 17, c;
c = a & b;
printf("%d", c);
a = 00000101 (1 3 4 2 1)
b = 00010001
o/p → 00000001

② e.g: a | b;
Bitwise OR
00000101
00010001
o/p → 00010101

0 ^ 0 = 0
0 ^ 1 = 1
1 ^ 0 = 1
1 ^ 1 = 0

③ Bitwise XOR

```
  0 0 0 0 0 1 0 1
^ 0 0 0 1 0 0 0 1
  0 0 0 1 0 1 0 0
```

④ Bitwise NOT

$(\sim a) = -(a+1)$ $\quad a = 5$

$= -(6)$

$= 1\ 1\ 1\ 0$

$\sim(a) = -(a+1)$ $\quad a = -5$

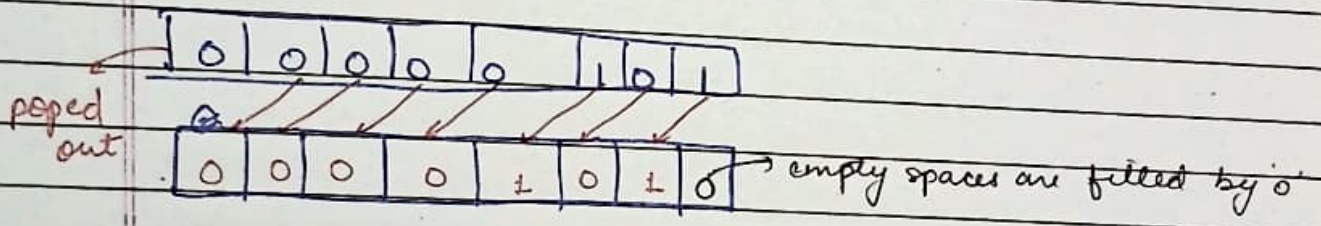
$\sim(a) = -(-5+1)$

$-(-4) \Rightarrow 4$

⑤ Bitwise left shift (<<)

5 << 1 → How many times

↓ left shift 5



⑥ Bitwise Right shift (>>)

RQ Right shift is like a division by 2.

10 ⇒ 5 ⇒ 2

Size of operator : It is a unary operator it gives result for compile time

```
int a = 10;
Size of(a);
```

4

```
char ch = 'A'
printf("%.d", size of ch);
```

of ①

float → ④

conditional operator: (? :)

exp 1

library f° → are predefined f° provided by C libraries that programmers can call to perform a common task

scanf() printf() strlen() sqrt() pow()

While loop

Syntax (expression/condition)

```
{
  statement;
}
```

```
{
S-1; // statement 1
S-2; // statement 2
S-3; // statement 3
```

E.g:
```
int i = 1
while (i<=5)
{
if ("%d", i);
i++;
}
```

Date
Page

do while

do while executes the body atleast once because the condition is checked after the body.

Syntax:

```
do {
    statements ;
    }
    while (expression / codition) ;
```

```
do {
    S1 ;
    S2 :
    S3 :
    } }
    while (expression/ condition);
```

```
int i = 10;
while (i<5)
  {
printf ("Hello")
  }
```

```
int i = 10;
do (i<5)
  {
if ("Hello")
  }
```

```
do {
printf ("Rahul");
  }
while (0);
```

O/P Falre

For Loop

Syntax

```
for ( initialization ; condition ; increment/decrement )
    { True
      code;
    }
```

```
for ( Roll=1 ; Roll<5 ; Roll ++)
    {
printf ("MCA");
    }
```

O/P
MCA
MCA
MCA
MCA

```
for (10; 0; 12)
    ↳ zero times
for ( 0; 1; 0)
    ↳ ∞ times
```

Nested loop:

Syntax:

```
for(initialization ; condition ; updation)
  {
   for(initialization ; condition ; updation)
   }
```

```
#include <stdio.h>
int main() {
for (i=1 ; i<=3 ; i++)
  {
  for (j=1 ; j<=4 ; j++)
     {
      if ("Pankaj",i=3 .j=4
     }
  }
}
```

Switch statement

Switch statement is a multiway used to execute one block based on the value of the expression.

Syntax:

```
switch (expression/condition) {
case constant 1: Block of statements
                 break;
case constant 14: Block of statements
                 break;
case constant 17: Block of statements
                 break;
default          Block of stat
                 break;
}
```

NOTE: Break is optional in switch statement.

Break: Immediately terminate the nearest loop for switch

```
for (int i=1; i<=10; i++)
{
if (i==5)
    break;
printf("%d", i);
}
```

Date Page — table

O/P

1 2 3 4 ✗

continue: Skips the remaining part of the current loop iteration & move to the next iteration

3 → Skip

```
for(int i=1; i<=5; i++) {
    if (i==3)) skip
    continue
    printf("%d", i);
}
```

1 2 4 5

goto statement - goto transfers program control to a labelled statement

```
goto end;
printf("Hello");
end
printf("end");
```

```
#include<stdio.h>
int main() {
int number;
number = 1;
repeat:
printf("%d \t", number);
number++;
if (number<=10)
goto repeat
}
return 0;
}
```

comma operation:

```
int x;
x = (2, 3, 4);
```

Right most → x=4

x = (7, 8, 9, 10, 11, 12)

x = 12

comma operator evaluates expression from left to right & the value of the complete comma expression is the right most value of the expression.

Date
Page

Q. WAP to print a no. is perfect no. or not

Q. WAP to check given no. is prime no. or

Q. WAP to find the no. of digits in the given no.

Q. WAP to print power of a no.

Q. WAP to check whether the no. is armstrong or not

Q. WAP to print the sum of factorial of digit a no.

Q.

String:

How do we store a student name is C.

Eg: char name[20];

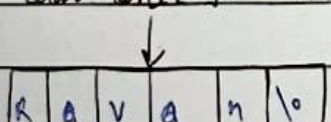
It is not an integer or floating point

sequence of character

String ① collection of symbols / characters

② Sequence of character terminated by null character

Eg: char name[ ] = "Ravan"



why we use '\0'

The null character marks the end of string

Imp → C does not have separate built in string data type

String are stored using character array.